

SIMATS ENGINEERING

ASSESSMENT TOOL 1 — MODEL ANSWERS

Course: Internet Programming (CSA43) | CO-1: HTML/XHTML, XML, Internet Protocols & HTTP

Question 1: Online Symposium Portal

Given code:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

1. Identify appropriate HTML5 semantic elements missing in the structure.

The snippet uses only a bare `<form>` with unlabeled inputs. It is missing several HTML5 semantic and accessibility elements:

- `<label>` — no label is associated with the "name" or "email" fields, so screen readers cannot announce what each field is for.
- `<fieldset>` and `<legend>` — used to group related form controls (e.g., "Registrant Details") and give the group a title.
- `<section>` / `<article>` — the registration form should sit inside a semantic `<section>` (e.g., a "Registration" section of the page) rather than floating directly in the body.
- `<header>` and `<main>` — the page lacks a `<header>` for the symposium title/logo and a `<main>` to wrap the primary content.
- `<button type="submit">` — the type attribute is not explicit, which can cause inconsistent behaviour across browsers.

2. Modify the structure to make it standards-compliant.

A standards-compliant, accessible version:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Symposium Registration</title>
</head>
<body>
  <header>
    <h1>College Symposium 2026</h1>
  </header>

  <main>
    <section id="registration">
      <h2>Registration Form</h2>
      <form action="/register" method="POST">
        <fieldset>
          <legend>Registrant Details</legend>

          <label for="name">Full Name:</label>
          <input type="text" id="name" name="name" required>

          <label for="email">Email Address:</label>
          <input type="email" id="email" name="email" required>

          <button type="submit">Submit</button>
        </form>
      </section>
    </main>
  </body>
</html>
```

```

        </fieldset>
    </form>
</section>
</main>

<footer>
    <p>&copy; 2026 College Symposium</p>
</footer>
</body>
</html>

```

3. Interpret the HTTP request generated during form submission.

Because method="POST" is used, when the student clicks Submit the browser constructs an HTTP POST request to the URL /register. The form field values are encoded (by default as application/x-www-form-urlencoded, e.g., name=John&email=john%40mail.com) and placed in the **body** of the request, not in the URL query string. A typical request looks like:

```

POST /register HTTP/1.1
Host: symposium.college.edu
Content-Type: application/x-www-form-urlencoded
Content-Length: 33

```

```
name=John+Doe&email=john%40mail.com
```

Key points: the request line specifies the POST method, the path, and the HTTP version; headers describe the content type and length; and the encoded form data forms the message body, keeping the data out of the visible URL and server logs.

4. Analyze the HTTP response cycle from server to browser.

Once the server (e.g., at /register) receives the POST request, it processes the submitted data (validation, database insertion, etc.) and returns an HTTP response. The general cycle is:

- 1) The server parses the request headers and body to extract name and email.
- 2) It performs server-side processing — validating fields, saving the record, checking duplicates.
- 3) It returns a status line, e.g., HTTP/1.1 200 OK (success) or HTTP/1.1 302 Found (redirect to a "Thank You" page), or an error such as 400 Bad Request / 500 Internal Server Error.
- 4) Response headers (Content-Type, Content-Length, Set-Cookie if a session is created) are sent, followed by the response body (an HTML confirmation page or JSON acknowledgement).
- 5) The browser receives the response, renders the new page (or follows the redirect), completing the request-response cycle.

5. Suggest improvements for usability and accessibility.

- Add <label> elements tied to each input via the for/id attributes so assistive technologies announce field purpose.
- Use the required and placeholder attributes, plus pattern/type validation (type="email") to give instant client-side feedback before the HTTP request is even sent.
- Provide clear error messages and aria-live regions for validation feedback, and aria-required="true" for mandatory fields.
- Ensure sufficient colour contrast and a visible focus outline for keyboard-only users tabbing through the form.
- Show a loading indicator or disable the submit button after click to prevent duplicate submissions while awaiting the server response.
- Confirm successful registration with a clear success message / redirect page, and use HTTPS to protect the submitted personal data in transit.

Question 2: Cross-Browser Compatibility Issue

Given code:

```
<html>
<head>
<title>Company</title>
</head>
<body>
<div>
<h1>Welcome
<p>About us
</div>
</body>
</html>
```

1. Identify improper nesting and missing closing tags.

- There is no `<!DOCTYPE html>` declaration, so the browser may render the page in "quirks mode" instead of standards mode.
- `<h1>Welcome` is never closed with `</h1>` before the `<p>` tag begins.
- `<p>About us` is never closed with `</p>`.
- The `<div>` is closed (`</div>`), but because the `<h1>` and `<p>` inside it were never closed, the parser must "auto-close" them implicitly, which can nest them incorrectly (e.g., the browser may end up placing `<p>` awkwardly relative to `<h1>`).
- No `lang` attribute on `<html>` and no `<meta charset>` in `<head>`, both of which are required for standards-compliant, accessible markup.

2. Analyze how different browsers may interpret this structure.

HTML parsers use error-correction ("tag soup" handling) rules defined in the HTML5 parsing algorithm, but different browsers/engines have historically applied slightly different heuristics for unclosed tags:

- Because the `<h1>` is not closed, most modern browsers (using the HTML5 parsing spec) will auto-close it as soon as the block-level `<p>` tag is encountered — but older or non-compliant parsers may instead nest `<p>` inside `<h1>`, producing invalid heading markup.
- Missing DOCTYPE pushes some browsers into quirks mode, which changes box-model calculations (e.g., how width/height/padding are computed), causing layout to differ from a standards-mode rendering in another browser.
- Whitespace and default margins around auto-closed elements can differ, so text spacing between "Welcome" and "About us" may look different across Chrome, Firefox, and Safari.
- Screen readers and SEO crawlers may also misinterpret the document outline, since the malformed heading/paragraph nesting breaks the semantic structure browsers rely on for consistent rendering.

3. Identify missing semantic elements (e.g., header, section).

- `<header>` — should wrap the site/company banner and the `<h1>` welcome heading.
- `<main>` — should wrap the primary page content.
- `<section>` — the "About us" text should be grouped in a semantic `<section>` with its own heading, e.g., `<h2>About Us</h2>`.
- `<footer>` — for copyright/contact information (not present at all).
- `<meta charset="UTF-8">` and `lang` attribute — needed inside `<head>`/`<html>` for correct text encoding and accessibility.

4. Provide a corrected W3C-compliant HTML structure.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Company</title>
</head>
<body>
  <header>
    <h1>Welcome</h1>
  </header>

  <main>
    <section id="about">
      <h2>About Us</h2>
      <p>About us</p>
    </section>
  </main>

  <footer>
    <p>&copy; 2026 Company Name</p>
  </footer>
</body>
</html>
```

5. Suggest techniques to ensure cross-browser compatibility.

- Always include `<!DOCTYPE html>` to force standards mode across all browsers.
- Validate markup using the W3C Markup Validation Service to catch unclosed/mis-nested tags before deployment.
- Use a CSS reset or `normalize.css` to neutralize default styling differences between browser engines.
- Use feature detection (e.g., via Modernizr or `@supports` in CSS) rather than browser-sniffing, and provide graceful fallbacks for unsupported features.
- Test on multiple real browsers/devices (Chrome, Firefox, Safari, Edge) or use cross-browser testing tools (e.g., BrowserStack).
- Use vendor prefixes where necessary for CSS and prefer well-supported, standards-based HTML5/CSS3 features over browser-specific hacks.

Question 3: Form Submission Failure

Given code:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

1. Identify issues in the form configuration.

- The `<button>` has `type="button"` instead of `type="submit"`. A `type="button"` element does not trigger form submission on click — it does nothing unless JavaScript explicitly calls `form.submit()`.
- The form method is `GET`, which is inappropriate for a login form because it appends field data (including the password) as plain text to the URL query string, e.g., `/submit?username=abc&password=xyz`.
- There is no name/id grouping, labels, or CSRF protection, and no HTTPS is enforced for the sensitive credential transfer.
- The password field's value would also be stored in browser history and server access logs because of the `GET` method, which is a serious security flaw.

2. Analyze why the HTTP request is not triggered.

A form is only submitted (an HTTP request generated) when a `<button>` or `<input>` of `type="submit"` inside the form is activated, or when JavaScript calls the form's `submit()` method. Here the button's `type` attribute is explicitly set to `"button"`, which per the HTML specification does **not** have any default form-submission behaviour — it is meant purely for custom JavaScript-driven actions (via an `onclick` handler). Since no script is attached to trigger submission, clicking "Login" does nothing, and no HTTP request is ever sent to `/submit`.

3. Interpret the role of GET method in this scenario.

Even if the button issue were fixed, `method="GET"` is unsuitable here. With `GET`, the browser encodes all form fields directly into the request URL's query string (e.g., `GET /submit?username=abc&password=xyz HTTP/1.1`) rather than in the request body. This means:

- Sensitive data (the password) becomes visible in the browser address bar, browser history, bookmarks, and server/proxy access logs.
- `GET` requests are meant to be idempotent/safe (retrieving data), and are commonly cached and pre-fetched by browsers — inappropriate for an action that changes server state such as authenticating a session.
- `GET` has practical URL-length limitations that could truncate data for larger forms.

4. Propose a corrected implementation.

```
<form action="/submit" method="POST">
  <label for="username">Username:</label>
  <input type="text" id="username" name="username" required>

  <label for="password">Password:</label>
  <input type="password" id="password" name="password" required>

  <button type="submit">Login</button>
</form>
```

Two changes fix the failure: (1) the button's `type` is changed to `"submit"` so clicking it triggers the form's default submission behaviour, and (2) the method is changed to `POST` so that the username and password are sent securely in the request body instead of the URL.

5. Suggest improvements for secure data transmission.

- Always serve the login page and submit the form over HTTPS (TLS) so credentials are encrypted in transit.
- Use method="POST" for any request that transmits sensitive data or changes server state.
- Hash and salt passwords on the server (e.g., bcrypt/Argon2) — never store or compare plaintext passwords.
- Add autocomplete="current-password" and appropriate input types, plus client-side and server-side validation.
- Implement CSRF tokens, rate limiting/account lockout on repeated failed attempts, and secure, HttpOnly, SameSite cookies for session management.
- Consider adding multi-factor authentication (MFA) for an additional layer of login security.